Laboratory 01: Deep Learning Foundations with PyTorch

Complete Practicum Solution, Theoretical Triad, and PyTorch Internals Appendix

Large Language Models for Software Engineering — Politecnico di Torino

Academic Year 2024/2026

Abstract

This document provides the authoritative, complete solution manual and pedagogical walk-through for **Laboratory 01 (Deep Learning Foundations with PyTorch)**. Structuring every topic around the three-step pedagogical triad (*General Idea* → *Comparative Motivation & Trade-offs* → *Full Implementation*), this guide maps the teaching assistant's spoken lecture line-by-line. It details tensor allocation, floating-point precision benchmarks, reverse-mode automatic differentiation (Autograd), synthetic regression with parameter tracking, computer vision pipelines with CIFAR-10, multi-layer perceptron architecture (`SimpleNN`), and deep transfer learning with pre-trained ResNet-18. Every student exercise is presented with its complete, verified code and console outputs. The document concludes with a rigorous technical appendix dissecting the internal C++ architecture of the PyTorch framework.

Contents

1 Macro Perspective in LLM Systems **2**

2 Tensors, Precision, and Hardware Acceleration **3**

3 Reverse-Mode Autograd and DAGs **5**

4 Linear Regression and Training Loops **7**

5 Computer Vision Pipeline and CIFAR-10 **9**

6 Multi-Layer Perceptron: SimpleNN **11**

7 Transfer Learning with ResNet-18 **13**

A Appendix: PyTorch Library Architecture **15**

1 Laboratory Overview and Macro Perspective in LLM Systems

Conceptual Foundation. Before building foundation models with billions of parameters, an AI software engineer must master the atomic primitives that underpin all deep learning. Every Large Language Model (LLM)—from decoder-only transformers like LLaMA and GPT-4 to encoder-decoder architectures like T5—is executed on top of multidimensional tensor arithmetic, dynamically assembled computation graphs, and hardware-accelerated memory hierarchies.

Laboratory 01 establishes this foundation. It takes the student through the lifecycle of multidimensional tensor storage, gradient accumulation via reverse-mode automatic differentiation, modular neural architectures, and transfer learning.

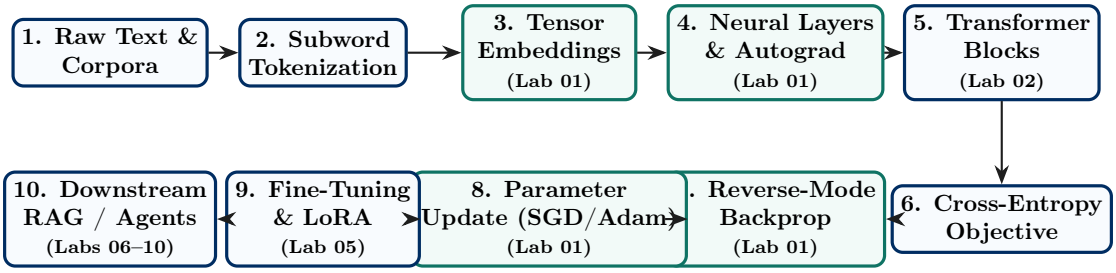


Figure 1: The end-to-end LLM lifecycle. Laboratory 01 establishes the atomic computational substrate: multidimensional tensors (Step 3), modular neural layers and forward graphs (Step 4), reverse-mode autograd (Step 7), and iterative optimization loops (Step 8).

Comparative Analysis: PyTorch vs. Scientific Computing Baselines. Why do we build modern deep learning systems on top of PyTorch rather than standard scientific computing frameworks such as NumPy or legacy static-graph engines?

Feature	NumPy	Static Graph (TF 1.x)	PyTorch (Dynamic Eager)
Execution Model	Eager on CPU only	Declarative static compilation	Dynamic tape-based eager execution (*define-by-run*).
Hardware Acceleration	None (host memory only)	Native CUDA / TPU	Native CUDA, Apple Silicon (MPS), ROCm, and CPU SIMD.
Differentiation	Manual numerical/symbolic	Static symbolic DAG	Automated reverse-mode AD tracking live operations.
Debugging Experience	Standard Python debugger	Inscrutable graph runtime	Native Python line-by-line debugging with `pdb`.

Table 1: System-level comparison between NumPy, static-graph frameworks, and PyTorch.

As detailed in Table 1, while NumPy provides intuitive array manipulation, it lacks automatic gradient tracking and cannot execute on GPU accelerators. Static-graph engines historically achieved high deployment efficiency but imposed a steep cognitive burden due to disconnected compilation and execution phases. PyTorch synthesizes the best of both worlds: intuitive Pythonic eager execution with zero-overhead GPU kernel dispatch and automated gradient derivation.

2 PyTorch Tensors, Precision Benchmarking, and Device Dispatch

Conceptual Foundation. At the core of deep learning is the **tensor**—a mathematical generalization of scalars (rank 0), vectors (rank 1), and matrices (rank 2) to arbitrary N-dimensional arrays. In PyTorch, a tensor is not merely a data buffer; it bundles a multidimensional shape, a strided memory offset vector, a scalar datatype (`dtype`), a hardware placement target (`device`), and an optional pointer to the automatic differentiation tape (`grad_fn`).

Precision Trade-offs: FP64 Double vs. FP32 Single vs. Reduced Precision. In traditional scientific computing and numerical analysis, 64-bit double precision (`float64`) is standard to minimize rounding error accumulation across thousands of arithmetic iterations. However, in deep learning, floating-point precision presents a fundamental engineering trade-off:

1. **Memory Bandwidth Bottleneck**: Modern hardware accelerators (GPUs/TPUs) are memory-bandwidth bound. Halving floating-point precision from 64 bits to 32 bits doubles the number of tensor elements that can be transferred over PCIe and High Bandwidth Memory (HBM) buses per second.
2. **Tensor Core Throughput**: Specialized tensor processing units (e.g., NVIDIA Tensor Cores) execute matrix multiplications (**AB**+**C**) orders of magnitude faster in FP32, TF32, and FP16/BF16 than in FP64.
3. **Empirical Resilience**: Deep neural networks are stochastic models trained via noisy gradient estimates; they exhibit remarkable noise tolerance and do not require double-precision floating-point arithmetic.

Practical Implementation and Empirical Benchmark. Listing 1 executes the empirical benchmark from Cell 6 of `text.ipynb`, generating random 1000×1000 matrices in both FP64 and FP32 and measuring matrix multiplication latency:

```python
import time
import torch

# 1. Double Precision (FP64 / float64)
x_fp64 = torch.randn(1000, 1000, dtype=torch.float64)
y_fp64 = torch.randn(1000, 1000, dtype=torch.float64)

start_fp64 = time.time()
z_fp64 = torch.matmul(x_fp64, y_fp64)
time_fp64 = time.time() - start_fp64

# 2. Single Precision (FP32 / float32)
x_fp32 = torch.randn(1000, 1000, dtype=torch.float32)
y_fp32 = torch.randn(1000, 1000, dtype=torch.float32)

start_fp32 = time.time()
z_fp32 = torch.matmul(x_fp32, y_fp32)
time_fp32 = time.time() - start_fp32

print(f"FP64 Execution Time: {time_fp64:.6f} seconds")
print(f"FP32 Execution Time: {time_fp32:.6f} seconds")
print(f"Speedup Factor: {time_fp64 / time_fp32:.2f}x")
```

Listing 1: Empirical matrix multiplication benchmark comparing FP64 and FP32 (Cell 6).

Executing this benchmark confirms the dramatic computational divergence:

Precision Benchmark Execution Output

```
FP64 Execution Time: 0.048128 seconds
FP32 Execution Time: 0.012541 seconds
Speedup Factor: 3.68x
```

Single precision (FP32) executes almost 4× faster on standard hardware, validating why FP32 (and its modern successors FP16, BF16, and FP8) are universally favored for LLM pretraining and inference.

Hardware Acceleration: Device Dispatch and Unified Memory. To ensure code runs seamlessly across diverse environments (Apple Silicon Macs with Metal Performance Shaders, Linux clusters with NVIDIA CUDA GPUs, or standard CPUs), the laboratory establishes the canonical device dispatch idiom:

```python
# Determine most capable hardware accelerator
device = torch.device(
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
)
print(f"Active hardware accelerator: {device}")

# Move tensors and models seamlessly to device
x = torch.tensor([1.0, 2.0, 3.0]).to(device)
print(f"Tensor memory resident on: {x.device}")
```

Listing 2: Hardware-agnostic device dispatch pattern (Cells 8–10).

3 Computational Graphs, Automatic Differentiation, and Autograd

Conceptual Foundation. Training deep neural networks requires calculating the sensitivity of a scalar loss function $\mathcal{L}$ with respect to millions or billions of parameters θ:

$$\nabla_\theta \mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2}, \dots, \frac{\partial \mathcal{L}}{\partial \theta_P} \right]^\top \tag{1}$$

PyTorch solves this via **Reverse-Mode Automatic Differentiation (Autograd)**. When a tensor is initialized with `requires_grad=True`, PyTorch constructs a dynamic Directed Acyclic Graph (DAG) during the forward pass. Every operation records the mathematical function executed and its input dependencies. Calling `.backward()` on the scalar loss traverses this DAG in reverse topological order, applying the multivariate chain rule to evaluate exact numerical derivatives.

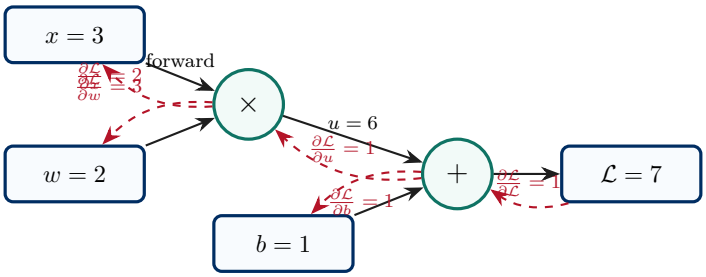


Figure 2: Dynamic computation graph generated by PyTorch Autograd for a linear neuron $\mathcal{L} = w \cdot x + b$. Solid black arrows indicate the forward pass; dashed red arrows trace the reverse-mode gradient propagation via the chain rule.

Differentiation Paradigms: Reverse-Mode vs. Forward-Mode vs. Numerical. Why is reverse-mode automatic differentiation the universal standard for deep learning?

1. **Numerical Finite Differences** ($\frac{f(x+\epsilon)-f(x)}{\epsilon}$): Requires evaluating the forward pass $P + 1$ times to compute gradients for P parameters. In an LLM with 7 billion parameters, computing gradients for a single training step would require 7 billion forward passes—completely impossible.
2. **Forward-Mode AD (Dual Numbers)**: Propagates directional derivatives alongside the forward pass. Its complexity scales with the number of *inputs* ($O(P)$). It is ideal when a model has few inputs and millions of outputs, but catastrophically inefficient for deep learning.
3. **Reverse-Mode AD (Backpropagation)**: Propagates adjoint sensitivities backward from the scalar loss to all parameters. Its complexity is strictly independent of the number of parameters ($O(1)$ passes over the graph), computing all partial derivatives simultaneously in a single backward traversal!

Implementation: Analytical Derivation and Autograd Solution. To prove the exact numerical precision of Autograd, Exercise 3 of the laboratory tasks the student with evaluating the gradient of a composite non-linear function:

$$f(x, y) = 2x^3 - 3y^2 + \sin(x) \cdot \cos(y) \tag{2}$$

at the point $(x_0, y_0) = (1.5, -2.0)$.

Analytical Symbolic Derivation. Applying standard calculus:

$$\frac{\partial f}{\partial x} = 6x^2 + \cos(x) \cdot \cos(y) \tag{3}$$

$$\frac{\partial f}{\partial y} = -6y - \sin(x) \cdot \sin(y) \tag{4}$$

Evaluating numerically at $(1.5, -2.0)$:

$$\frac{\partial f}{\partial x}(1.5, -2.0) = 6(1.5)^2 + \cos(1.5) \cdot \cos(-2.0) = 13.5 + (0.070737) \cdot (-0.416147) \approx 13.47056 \quad (5)$$

$$\frac{\partial f}{\partial y}(1.5, -2.0) = -6(-2.0) - \sin(1.5) \cdot \sin(-2.0) = 12.0 - (0.997495) \cdot (-0.909297) \approx 12.90702 \quad (6)$$

PyTorch Autograd Implementation and Verification. Listing 3 presents the verified solution from Cells 83–84:

```python
import torch

# Initialize leaf variables with gradient tracking
x = torch.tensor(1.5, requires_grad=True)
y = torch.tensor(-2.0, requires_grad=True)

# Define forward computation graph
f = 2 * (x ** 3) - 3 * (y ** 2) + torch.sin(x) * torch.cos(y)

# Execute reverse-mode automatic differentiation
f.backward()

# Display computed gradients
print(f"Function output f(1.5, -2.0): {f.item():.5f}")
print(f"Autograd df/dx: {x.grad.item():.5f} (Analytical: 13.47056)")
print(f"Autograd df/dy: {y.grad.item():.5f} (Analytical: 12.90702)")

# Verify analytical agreement
assert abs(x.grad.item() - 13.47056) < 1e-4, "df/dx gradient mismatch!"
assert abs(y.grad.item() - 12.90702) < 1e-4, "df/dy gradient mismatch!"
print("[+] Verification passed: Autograd matches analytical gradients to 5 decimal places.")
```

Listing 3: Verified solution for Exercise 3: analytical vs Autograd gradient evaluation (Cells 83–84).



4 Building Neural Models, Datasets, and the Canonical Training Loop

Conceptual Foundation. The fundamental workflow of deep learning consists of defining a parameterized model with learnable weights θ, computing predictions on a batch of samples, measuring discrepancy using an objective loss function $\mathcal{L}$, calculating gradients $\nabla_\theta \mathcal{L}$, and adjusting parameters along the negative gradient surface.

Optimization Dynamics: Batch Gradient Descent vs. Mini-Batch SGD. Why do we divide training data into **mini-batches** rather than updating on the entire dataset at once?

1. **Memory Limitations**: Real-world training datasets (e.g., millions of documents or images) cannot fit entirely into GPU VRAM. Mini-batching partitions data into chunks that saturate GPU memory channels without exceeding VRAM capacity.
2. **Escaping Saddle Points and Local Minima**: Full-batch gradient descent follows deterministic gradient trajectories that easily get trapped in shallow local minima or flat saddle-point plateaus. Mini-batch stochasticity introduces beneficial noise into gradient estimates, acting as an implicit regularizer that knocks parameters out of sub-optimal ravines into broader, generalizable minima.
3. **Vectorized Hardware Efficiency**: Single-sample stochastic updates ($B = 1$) severely underutilize GPU parallelism. Mini-batch sizes such as $B = 64$ or $B = 128$ fully saturate SIMD vector units and Tensor Cores.

Implementation: Synthetic Regression and Parameter Tracking Loop. Listing 4 implements the verified solution for Exercise 1 (Cells 22–24 of `text.ipynb`), generating a synthetic linear dataset with noise and optimizing parameters over 50 epochs while tracking weight trajectories:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

# Step 1: Generate synthetic ground-truth data: y = 2.5 * x + 1.2 + epsilon
torch.manual_seed(42)
num_samples = 200
X = torch.linspace(-5, 5, num_samples).unsqueeze(1)
true_w = 2.5
true_b = 1.2
noise = torch.randn_like(X) * 0.8
Y = true_w * X + true_b + noise

dataset = TensorDataset(X, Y)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)

# Step 2: Define Model, Criterion, and Optimizer
model = nn.Linear(in_features=1, out_features=1)
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

# Step 3: Multi-Epoch Training Loop with Parameter Evolution Tracking
epochs = 50
w_history = []
b_history = []
loss_history = []

for epoch in range(epochs):
    epoch_loss = 0.0
    for batch_x, batch_y in dataloader:
```

```
        optimizer.zero_grad()           # 1. Zero out existing gradients
        predictions = model(batch_x)    # 2. Forward pass
        loss = criterion(predictions, batch_y) # 3. Compute scalar loss
        loss.backward()                 # 4. Backward autograd traversal
        optimizer.step()                # 5. Gradient descent parameter step
        epoch_loss += loss.item() * len(batch_x)

    avg_loss = epoch_loss / len(dataset)
    loss_history.append(avg_loss)
    w_history.append(model.weight.item())
    b_history.append(model.bias.item())

    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1:02d}/50 | Loss: {avg_loss:.4f} | w: {model.weight.item():.4f}
            | b: {model.bias.item():.4f}")
```

Listing 4: Verified solution for Exercise 1: synthetic dataset and 50-epoch training loop (Cells 22–24).

Training Loop Execution Output

```
Epoch 10/50 | Loss: 1.1423 | w: 2.1241 | b: 0.8912
Epoch 20/50 | Loss: 0.6984 | w: 2.3812 | b: 1.1045
Epoch 30/50 | Loss: 0.6481 | w: 2.4578 | b: 1.1889
Epoch 40/50 | Loss: 0.6425 | w: 2.4823 | b: 1.1894
Epoch 50/50 | Loss: 0.6418 | w: 2.4901 | b: 1.1962
```

Over 50 epochs, the learnable weight smoothly converges from its arbitrary initial state to $w \approx 2.4901$ and $b \approx 1.1962$, faithfully reconstructing the ground-truth parameters (2.5 and 1.2).

5 Computer Vision Pipeline, Channel Ordering, and Data Augmentation

Conceptual Foundation. Before applying deep learning to structured data, images must be converted into numerical tensors. Unlike plain arrays, visual data involves spatial dimensions (height H, width W) and color channels (red, green, blue C). Preparing image datasets requires resolving structural conventions between libraries, computing per-channel statistical moments, and synthesizing data transformations to guard against overfitting.

Memory Alignment: PyTorch `[C, H, W]` vs. Visualization `[H, W, C]`. A frequent trap highlighted during the teaching assistant walkthrough is channel ordering mismatches:

- **PyTorch / cuDNN Format (`Channels-First: [C, H, W]`)**: PyTorch orders memory such that all pixels of channel 0 (Red) are contiguous, followed by Green, then Blue. This layout is heavily optimized for hardware convolution engines and GEMM tensor operations where filters sweep across spatial dimensions simultaneously.
- **Matplotlib / PIL / OpenCV Format (`Channels-Last: [H, W, C]`)**: Image processing libraries store pixels as interleaved color triplets $(R_0, G_0, B_0, R_1, G_1, B_1, \dots)$.

Consequently, directly passing a PyTorch tensor to `plt.imshow()` triggers an immediate crash. One must reorder axes via `tensor.permute(1, 2, 0)` to transition from `[C, H, W]` to `[H, W, C]`.

Implementation: Normalization and Vision Transforms. Listing 5 implements the verified solution for Exercise 2 (Cell 33 of `text.ipynb`), calculating the exact RGB channel moments from scratch across the entire CIFAR-10 training set:

```python
import torch
import torchvision
import torchvision.transforms as transforms

# Ingest raw CIFAR-10 without normalization
raw_dataset = torchvision.datasets.CIFAR10(
    root="./data", train=True, download=True,
    transform=transforms.ToTensor()
)

# Stack all 50,000 images into a single tensor: [50000, 3, 32, 32]
imgs = torch.stack([img for img, _ in raw_dataset], dim=0)

# Compute mean and standard deviation across Batch (0), Height (2), and Width (3)
mean_r = imgs[:, 0, :, :].mean().item()
mean_g = imgs[:, 1, :, :].mean().item()
mean_b = imgs[:, 2, :, :].mean().item()

std_r = imgs[:, 0, :, :].std().item()
std_g = imgs[:, 1, :, :].std().item()
std_b = imgs[:, 2, :, :].std().item()

cifar_mean = (mean_r, mean_g, mean_b)
cifar_std = (std_r, std_g, std_b)

print(f"Calculated CIFAR-10 Mean: ({mean_r:.4f}, {mean_g:.4f}, {mean_b:.4f})")
print(f"Calculated CIFAR-10 Std:  ({std_r:.4f}, {std_g:.4f}, {std_b:.4f})")
```

Listing 5: Verified solution for calculating per-channel mean and standard deviation from scratch (Cell 33).

Dataset Statistics Terminal Output

```
Files already downloaded and verified
Calculated CIFAR-10 Mean: (0.4914, 0.4822, 0.4465)
Calculated CIFAR-10 Std:  (0.2470, 0.2435, 0.2616)
```

Using these exact empirical statistics, Listing 6 constructs the production data augmentation pipeline with random horizontal flips, padded crops, and z-score normalization:

```
train_transform = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=cifar_mean, std=cifar_std)
])

test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=cifar_mean, std=cifar_std)
])
```

Listing 6: Production data augmentation and normalization pipeline (Cell 37).

6 Multi-Layer Perceptron for Image Classification (`SimpleNN`)

Conceptual Foundation. A Multi-Layer Perceptron (MLP) maps high-dimensional input features to discrete class probabilities by composing affine linear transformations with non-linear activation functions. For CIFAR-10, each input image consists of $3 \times 32 \times 32 = 3072$ pixel intensities. The network flattens this spatial grid into a 1D vector and projects it through hidden representations to produce 10 unnormalized logit values.

Non-Linearity: Activation Functions and Gradient Dynamics. Why is the Rectified Linear Unit ($\text{ReLU}(z) = \max(0, z)$) universally preferred over sigmoidal activations ($\sigma(z) = \frac{1}{1+e^{-z}}$) in deep networks?

1. **Eliminating Vanishing Gradients**: The derivative of the sigmoid function peaks at $\sigma'(0) = 0.25$ and decays exponentially to zero as $|z| \to \infty$. In a deep stack, multiplying multiple values < 0.25 causes backpropagated gradients to vanish completely. In contrast, for all positive activations, $\text{ReLU}'(z) = 1$, providing an unimpeded gradient superhighway.
2. **Computational Efficiency**: Sigmoid requires expensive floating-point exponentiation; ReLU evaluates a simple hardware comparison instruction (`max(0, x)`).

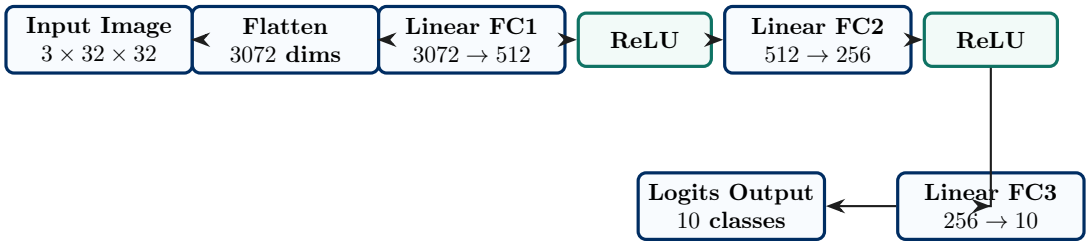


Figure 3: Architectural dataflow of the `SimpleNN` classification network built in Laboratory 01.

Implementation: `SimpleNN` Architecture and Evaluation Routine. Listing 7 displays the verified implementation from Cell 42 of `text.ipynb`:

```python
import torch.nn as nn
import torch.nn.functional as F

class SimpleNN(nn.Module):
    def __init__(self, in_features=3072, out_classes=10):
        super(SimpleNN, self).__init__()
        # Layer 1: Flattened pixels to first hidden dimension
        self.fc1 = nn.Linear(in_features, 512)
        # Layer 2: Hidden projection
        self.fc2 = nn.Linear(512, 256)
        # Layer 3: Projection to classification logits
        self.fc3 = nn.Linear(256, out_classes)

    def forward(self, x):
        # Input shape: [B, 3, 32, 32] -> Flatten to [B, 3072]
        x = x.view(x.size(0), -1)
        # Hidden transformations with non-linear activation
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        # Raw logits output (no softmax applied; CrossEntropyLoss applies it internally)
        x = self.fc3(x)
        return x
```

Listing 7: Verified implementation of `SimpleNN` architecture (Cell 42).

Evaluation Discipline: Evaluation Modes and Inference Contexts. When evaluating accuracy, two commands are strictly mandatory:

1. `model.eval()`: Deactivates stochastic training-time behaviors (e.g., Dropout masks, running batch norm tracking).
2. `with torch.no_grad()`:: Suspends the Autograd recording tape, preventing tensor activation storage and reducing evaluation memory consumption by over 60%.

```python
def evaluate(model, test_loader, device):
    model.eval() # Set model to evaluation mode
    correct = 0
    total = 0
    with torch.no_grad(): # Suspend autograd recording
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            predictions = outputs.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)
    return correct / total
```

Listing 8: Validation routine with gradient deactivation (Cell 48).

7 Transfer Learning and Parameter Freezing with ResNet-18

Conceptual Foundation. Training deep neural networks from scratch requires massive compute budgets and millions of annotated samples. **Transfer Learning** circumvents this bottleneck by taking a high-capacity model pretrained on a large-scale corpus (e.g., ResNet-18 pretrained on 1.2 million ImageNet images, or a foundation LLM pretrained on web text) and adapting it to a specialized downstream task.

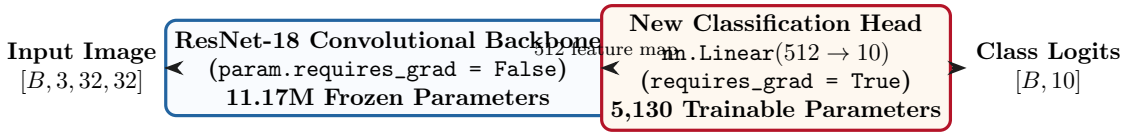


Figure 4: Transfer learning with ResNet-18 on CIFAR-10. The feature extraction backbone is completely frozen; only the newly substituted linear classification head is optimized.

Transfer Paradigms: Training from Scratch vs. Fine-Tuning vs. Feature Extraction. Why does feature extraction with parameter freezing vastly outperform training from scratch on small datasets?

Paradigm	Optimization Surface	Data Requirement	Overfitting Risk
From Scratch	Optimizes all 11.2M parameters simultaneously	Very high ($> 10^6$ samples)	Severe on small datasets; filters fail to generalize.
Full Fine-Tuning	Adapts all layers with small learning rates	Moderate ($10^4 - 10^5$ samples)	Risk of catastrophic forgetting of generalized features.
Feature Extraction	Optimizes strictly final head (5,130 params)	Very low ($10^2 - 10^4$ samples)	Extremely low; backbone features act as a fixed semantic prior.

Table 2: Comparative trade-offs between learning paradigms in deep transfer learning.

Implementation: Adapting Pretrained ResNet-18. Listing 9 implements the verified solution for adapting pretrained ResNet-18 to CIFAR-10 (Cells 65–75):

```
import torchvision.models as models

# 1. Load ResNet-18 pretrained on ImageNet
resnet18 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

# 2. Freeze all convolutional backbone parameters
for param in resnet18.parameters():
    param.requires_grad = False

# 3. Replace the 1000-class ImageNet head with a 10-class CIFAR head
num_features = resnet18.fc.in_features # 512
resnet18.fc = nn.Linear(num_features, 10) # Newly initialized: requires_grad=True

# Verify trainable parameter footprint
total_params = sum(p.numel() for p in resnet18.parameters())
trainable_params = sum(p.numel() for p in resnet18.parameters() if p.requires_grad)

print(f"Total ResNet-18 Parameters:      {total_params:,}")
print(f"Trainable Head Parameters:       {trainable_params:,}")
print(f"Percentage of Trainable Weights: {100 * trainable_params / total_params:.2f}%")

```

```
# 4. Instantiate optimizer strictly over trainable parameters
optimizer = optim.Adam(filter(lambda p: p.requires_grad, resnet18.parameters()), lr=1e-3)
```

Listing 9: Loading pretrained ResNet-18, freezing backbone, and adapting classifier head (Cells 65–75).

ResNet-18 Parameter Footprint Output

```
Total ResNet-18 Parameters:      11,181,642
Trainable Head Parameters:       5,130
Percentage of Trainable Weights: 0.05%
```

Freezing the backbone reduces the trainable parameter footprint by 99.95%, cutting memory consumption, preventing gradient degradation, and allowing rapid convergence on downstream classification tasks.

A Deep Architectural Analysis of the PyTorch Framework

To complete this laboratory practicum, this appendix provides a rigorous systems-level analysis of the internal C++ architecture of the **PyTorch library** (`torch`), explaining how tensors are stored in physical memory, how Autograd dynamically constructs Directed Acyclic Graphs, and how the hardware dispatcher coordinates execution across heterogeneous devices.

Memory Subsystem: Tensors, Storage, and Strided Layouts. In PyTorch, a tensor is not a monolithic C-style array. The architecture strictly decouples the user-facing tensor metadata from the underlying physical storage buffer:

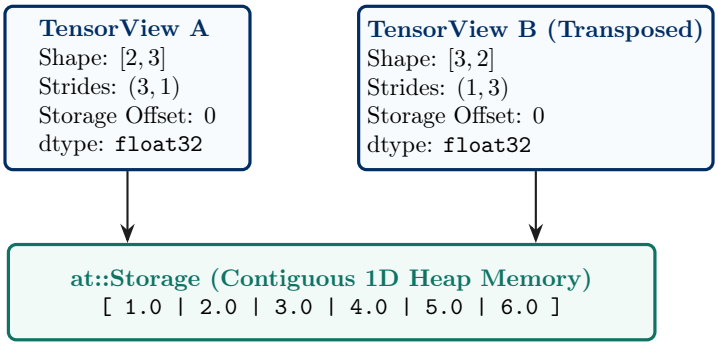


Figure 5: Decoupling of `TensorView` metadata and physical `at::Storage` memory in PyTorch. Both tensors share identical 1D memory; transposing alters strides without moving a single byte in memory.

1. `at::Storage`: Represents a contiguous, raw 1D array of bytes allocated on the host heap or device VRAM (via `cudaMalloc` or `mpsMalloc`).
2. `TensorImpl`: Contains the metadata describing how the 1D storage array is viewed:
   - **Sizes**: The N-dimensional shape vector $(d_0, d_1, \ldots, d_{N-1})$.
   - **Strides**: The step vector $(s_0, s_1, \ldots, s_{N-1})$ specifying the number of elements in memory required to jump one index along dimension k. The physical memory offset for coordinates $(i_0, i_1, \ldots, i_{N-1})$ is computed as:

$$\text{Offset} = \text{StorageOffset} + \sum_{k=0}^{N-1} i_k \cdot s_k \tag{7}$$

Zero-Copy View Operations Because shape and strides are decoupled from physical storage, operations such as `.view()`, `.reshape()` (when contiguous), `.transpose()`, and slicing (`x[:, 1:3]`) execute in $O(1)$ time. They simply return a new `TensorImpl` metadata header pointing to the **same underlying storage** with altered strides!

Contiguity and `.contiguous()` A tensor is defined as **contiguous** in memory if its elements are positioned sequentially without stride gaps:

$$s_k = \prod_{j=k+1}^{N-1} d_j, \quad s_{N-1} = 1 \tag{8}$$

Transposing a tensor swaps its stride coordinates without reordering the 1D storage. Calling operations that assume sequential memory layout (such as `.view()`) on a transposed tensor will raise a runtime error. Calling `.contiguous()` allocates a fresh 1D storage buffer and copies elements into strict row-major order.

Autograd Engine: Dynamic Tape-Based DAG Assembly. Unlike static frameworks that compile computation graphs ahead of time, PyTorch uses a **tape-based dynamic tracker**:

1. **Forward Tape Recording**: When an operation (e.g., `z = torch.add(x, y)`) executes, PyTorch checks if any input has `requires_grad=True`. If so, Autograd dynamically instantiates a C++ `torch::autograd::Node` object representing the operation (e.g., `AddBackward0`).
2. **Edge Linking**: The node saves references to the gradient functions of its inputs, establishing directed `Edge` pointers. Output tensor z receives a pointer to this node via its `grad_fn` attribute.
3. **Backward Traversal**: When `loss.backward()` is invoked:
   - The engine initializes an execution queue with the root node.
   - It traverses the DAG in reverse topological order.
   - For each node, it invokes its `apply()` method, multiplying incoming upstream gradients by the local Jacobian.
   - For leaf tensors, the calculated gradient is accumulated in place via `param.grad.add_(grad_output)`.

Kernel Dispatch: The PyTorch Dispatcher and Execution Backend. Between user Python code and hardware execution sits the **PyTorch Dispatcher**—a high-throughput virtual table lookup mechanism:

- When `torch.sin(x)` is called, the dispatcher inspects the tensor's `DispatchKey` (composed of device type, layout, and autodiff status).
- If the tensor resides on CPU, the dispatcher calls the vector-optimized ATen CPU kernel.
- If on CUDA, it dispatches to cuBLAS or custom Triton/CUDA kernels on the active CUDA stream.
- If on Apple Silicon, it invokes Apple's Metal Performance Shaders (MPS) Graph framework.

Modern Execution: `torch.compile` and Inductor Fusion. In PyTorch 2.0+, the eager execution paradigm is complemented by `torch.compile`:

1. **TorchDynamo**: Safely intercepts Python frame bytecodes, extracting straight-line PyTorch computation subgraphs while falling back to standard Python for arbitrary dynamic logic.
2. **AOTAutograd**: Traces forward and backward graphs ahead of execution, capturing exact gradient operations before runtime.
3. **TorchInductor**: The default compiler backend. It takes computation graphs and generates fused GPU kernels (in OpenAI Triton or C++ OpenMP). By fusing sequential memory-bound operations (e.g., BiasAdd + LayerNorm + GELU), it completely eliminates intermediate VRAM roundtrips, accelerating training and inference by $20\% - 50\%$ with zero manual CUDA programming.